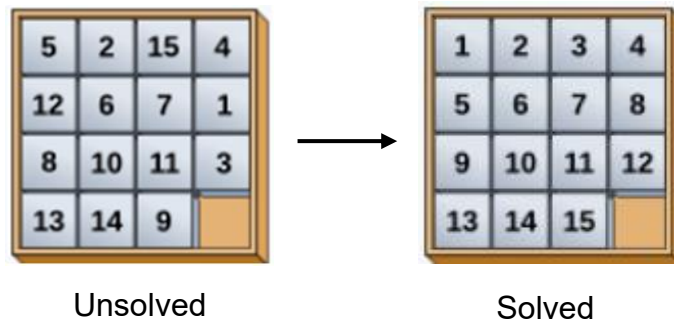


3 Moving Hole Puzzle

A moving hole puzzle, also known as a sliding puzzle, involves moving pieces around a board to achieve a specific configuration. The puzzle typically has one empty space (the "hole") that allows the pieces to be slid into different positions.



Task 3.1

Write a function `create_puzzle(n)` that:

- takes an integer value `n`
- creates a 2D list of `n x n` dimension
- inserts the numbers from 0 to n^2-1 into the 2D list at **random** positions
- identifies the position of number 0 which represents the missing tile of the puzzle
- returns the 2D list, the row and column index of the number 0 (the missing tile) in a list.

[4]

For example,

```
>>> puzzle, r, c = create_puzzle(4)
>>> print(puzzle)
[[14, 10, 9, 3], [2, 4, 5, 12], [13, 0, 15, 8], [7, 1, 6, 11]]
>>> print(r, c)
2, 1
>>> puzzle[r][c]
0
```

Test your function using the following:

[1]

```
for i in range(3, 6):
    puzzle, r, c = create_puzzle(i)
    print("Coordinate of space: ({} , {})".format(r, c))
    print("Puzzle: " + str(puzzle))
    print()
```

Task 3.2

Write a function `display_puzzle(puzzle)` that:

- takes a 2D list which represents the puzzle, `puzzle`
- displays the puzzle with the row and col index as shown in the example below for easy reference. Take note that the `#` character is used to represent the missing tile.

[3]

For example,

```
>>> puzzle = [[14, 10, 9, 3], [2, 4, 5, 12], [13, 0, 15, 8],
[7, 1, 6, 11]]
>>> display_puzzle(puzzle)
```

	0	1	2	3
0	14	10	9	3
1	2	4	5	12
2	13	#	15	8
3	7	1	6	11

Task 3.3

The available moves that the player can make depends on the position of the missing tile in the puzzle. There are 4 moves altogether.

- 'U' means move the tile above the missing tile down
- 'R' means move the tile on the right of the missing tile to the left
- 'D' means move the tile below the missing tile up
- 'L' means move the tile on the left of the missing tile to the right

[14][10][9][3]

[2][4][5][12]

[13][][15][8]

[7][1][6][11]

U move

----->

[14][10][9][3]

[2][][5][12]

[13][4][15][8]

[7][1][6][11]

[14][10][9][3]

[2][4][5][12]

[13][][15][8]

[7][1][6][11]

R move

----->

[14][10][9][3]

[2][4][5][12]

[13][15][][8]

[7][1][6][11]

[RVHS/2024/Q3 Cont']

[14][10][9][3]		[14][10][9][3]
[2][4][5][12]	D move	[2][4][5][12]
[13][][15][8]	----->	[13][1][15][8]
[7][1][6][11]		[7][][6][11]

[14][10][9][3]		[14][10][9][3]
[2][4][5][12]	L move	[2][4][5][12]
[13] [][15][8]	----->	[][13][15][8]
[7][1][6][11]		[7][1][6][11]

Take note that not all positions of the missing tile will have all 4 moves mentioned above. If the missing tile is at the edge of the puzzle, it will only have 3 available moves. If it is at the corner, it will only have 2.

Write a function `find_available_move(puzzle, r, c)` that:

- takes a 2D list which represent the puzzle, `puzzle`
- takes two integer values `r` and `c` which represent the row and column index of the missing tile.
- returns a list of moves that is available given the missing tile is at `puzzle[r][c]`.
- You can assume the `r` and `c` are always valid.

[3]

For example,

```
>>> puzzle = [[14, 10, 9, 3], [2, 4, 5, 12], [13, 0, 15, 8],
[7, 1, 6, 11]]
>>> find_available_move(puzzle, 2, 1)
['U', 'R', 'D', 'L']
```

Task 3.4

Write a function `make_move(move, puzzle, r, c)` that:

- takes a string value `move` which has either the values of 'U', 'R', 'D' or 'L'
- takes a 2D list which represent the puzzle, `puzzle`
- takes two integer values `r` and `c` which represent the row and column index of the missing tile.
- executes the move in the `puzzle` by swapping the relevant tiles
- you can assume that the move is always valid.

[4]

For example,

```
>>> puzzle = [[14, 10, 9, 3], [2, 4, 5, 12], [13, 0, 15, 8],
[7, 1, 6, 11]]
>>> make_move('D', puzzle, 2, 1)
>>> print(puzzle)
[[14, 10, 9, 3], [2, 4, 5, 12], [13, 1, 15, 8], [7, 0, 6, 11]]
```

Task 3.5

Write a function `win(puzzle)` that:

- takes a 2D list which represent the puzzle, `puzzle`
- returns `True` if `puzzle` is solved and `False` otherwise.
- take note that square puzzle size $n \times n$ can vary.

[3]

Task 3.6

Write a function `game_menu()` that:

- starts the game with a default puzzle using `[[2, 3, 5], [1, 0, 4], [7, 8, 6]]`
- starts the timer when the game is launched
- displays the puzzle and the option menu at the start of the game and after every move made by the player
- implement the following menu options:
 - 1) New game
 - prompts user to choose a puzzle size from 3 to 5 and generates a new puzzle when option 1 is chosen
 - restarts the timer
 - 2) Make one move
 - prompts user to make one valid move by displaying all valid moves based on the position of the missing tile when option 2 is chosen
 - executes the move
 - 3) Make many moves
 - prompts user to make a series of moves (e.g. RULLDRRD) when option 3 is chosen
 - executes the moves
 - reverts the puzzle to the original state if the series of moves contains invalid moves
 - 4) Quit
 - displays "You took <x>s to give up." where x is the time taken for the player to give up
 - quits the game
- checks for winning condition after every move
- displays the total time taken to win the game in second when the game is won
- all data validation must be present

[10]

[RVHS/2024/Q3 Cont']

A sample display of the game is as follow:

Sample run A – demonstrate option 2) and 3)

```

      0    1    2

0      2    3    5
1      1    #    4
2      7    8    6
1) New Game.
2) Make one Move.
3) Make many moves.
4) Quit.
Your option: 2
Please choose your move ['U', 'R', 'D', 'L']: R

      0    1    2

0      2    3    5
1      1    4    #
2      7    8    6
1) New Game.
2) Make one Move.
3) Make many moves.
4) Quit.
Your option: 2
Please choose your move ['U', 'D', 'L']: U

      0    1    2

0      2    3    #
1      1    4    5
2      7    8    6
1) New Game.
2) Make one Move.
3) Make many moves.
4) Quit.
Your option: 3
Your series of moves (no spaces): LLDRRD
You took 15s to win.
```

[RVHS/2024/Q3 Cont']

Sample run B – demonstrate option 1) and 4)

```

      0    1    2
0      2    3    5
1      1    #    4
2      7    8    6
1) New Game.
2) Make one Move.
3) Make many moves.
4) Quit.
Your option: 1
Key in size of puzzle: 5

      0    1    2    3    4
0      2    8   20   15   16
1     14   17   21    1   18
2      3   12   23    7   11
3     13    9    6   10   22
4      4   24    #   19    5
1) New Game.
2) Make one Move.
3) Make many moves.
4) Quit.
Your option: 4
Game over.
You took 2s to give up.
```